

섹션 01

Subagents & HITL

위임과 승인으로 안전한 Deep Agent 만들기



SECTION 01 · SUBAGENTS & HITL

§0. 2주차 목표와 큰 그림

1주차가 Deep Agents의 전체 지도를 그렸다면, 2주차는 그 지도에서 **서브에이전트**와 **Human-in-the-loop** 두 지점을 깊게 파고든다. 두 기능은 서로 다른 문제를 해결한다. 서브에이전트는 메인 에이전트의 컨텍스트가 커지는 문제를 줄이고, Human-in-the-loop은 위험한 도구 호출이 실제로 실행되기 전에 사람의 판단을 끼워 넣는다.

이 글의 한 줄 요약은 다음과 같다.

서브에이전트는 컨텍스트를 격리하고, Human-in-the-loop은 위험한 실행을 사람 승인으로 제어한다.

발표의 흐름은 위임에서 시작해 승인으로 끝난다. 먼저 메인 에이전트가 언제 하위 작업을 넘겨야 하는지 살펴보고, `SubAgent` 딕셔너리의 필드를 하나씩 확인한다. 그 다음 `interrupt_on` 과 체크포인트를 붙여 도구 호출을 멈추고, 사람이 `approve` , `edit` , `reject` 중 하나를 선택해 같은 `thread_id` 로 재개하는 흐름을 만든다.



§1. 왜 서브에이전트인가

서브에이전트의 핵심 가치는 **컨텍스트 격리**다. 웹 검색, 파일 읽기, 데이터 분석처럼 중간 결과가 많은 작업은 메인 에이전트의 메시지 이력을 빠르게 부풀린다. 메인 에이전트가 모든 검색 결과와 중간 판단을 직접 들고 있으면, 중요한 시스템 프롬프트와 사용자 요구가 뒤로 밀리고 최종 판단이 흐려진다.

서브에이전트는 이 무거운 작업을 별도의 컨텍스트에서 수행한다. 메인 에이전트는 `task` 도구를 통해 하위 작업을 맡기고, 서브에이전트는 자체 도구와 지침으로 일을 끝낸 뒤 최종 요약만 반환한다. 그래서 메인 컨텍스트에는 세부 도구 호출 기록이 아니라, 다음 결정을 내릴 수 있는 압축된 결과만 남는다.

서브에이전트를 쓰기 좋은 경우는 세 가지다.

상황	이유
검색·파일 읽기처럼 출력이 큰 작업	중간 결과를 메인 컨텍스트 밖에 둔다
전문 지침이 필요한 작업	서브에이전트별 시스템 프롬프트를 분리한다
다른 도구 세트가 필요한 작업	필요한 도구만 주어 선택지를 줄인다

반대로 단순한 단일 단계 작업에는 서브에이전트가 오버헤드가 될 수 있다. 위임은 공짜가 아니므로, 메인 에이전트가 직접 처리해도 컨텍스트가 깨끗하게 유지되는 작업은 그대로 두는 편이 좋다.

§2. SubAgent 구성

대부분의 예제에서는 서브에이전트를 딕셔너리로 정의한다. 필수 필드는 네 개다.

SubAgent Dictionary

name: 고유 식별자

description: 라우팅 힌트

system_prompt: 작업 지침

tools: 사용 가능한 도구

model

middleware

interrupt_on

필드	필수	의미
name	O	메인 에이전트가 <code>task()</code> 호출 때 사용하는 고유 이름
description	O	메인 에이전트가 언제 위임할지 판단하는 설명
system_prompt	O	서브에이전트의 역할, 도구 사용법, 출력 형식
tools	O	서브에이전트가 쓸 수 있는 도구 목록
model	선택	메인과 다른 모델을 쓰고 싶을 때 지정
middleware	선택	서브에이전트에 별도 미들웨어를 붙일 때 사용
interrupt_on	선택	서브에이전트 내부 도구 승인 정책

가장 중요한 필드는 `description` 과 `system_prompt` 다. `description` 은 메인 에이전트의 라우팅 힌트이고, `system_prompt` 는 실제 작업 품질을 결정한다. 설명이 “Does research”처럼 모호하면 메인 에이전트가 언제 호출해야 할지 판단하기 어렵다. “웹 검색으로 기술 주제를 조사하고, 근거 URL과 함께 500단어 이하 요약을 반환한다”처럼 행동과 출력이 드러나야 한다.

```
research_subagent = {
    "name": "research-agent",
    "description": "Deep Agents 개념을 짧게 조사하고 핵심만 요약한다.",
    "system_prompt": "도구 결과를 길게 복사하지 말고 5문장 이하 요약만 반환한다.",
    "tools": [summarize_notes],
}
```

§3. 일반적인 서브에이전트 패턴

첫 번째 패턴은 **범용 서브에이전트**다. deepagents는 사용자 정의 서브에이전트 외에도 `general-purpose` 서브에이전트를 사용할 수 있다. 전문 도구나 특별한 지침은 필요 없지만, 메인 컨텍스트를 깨끗하게 유지하고 싶을 때 적합하다.

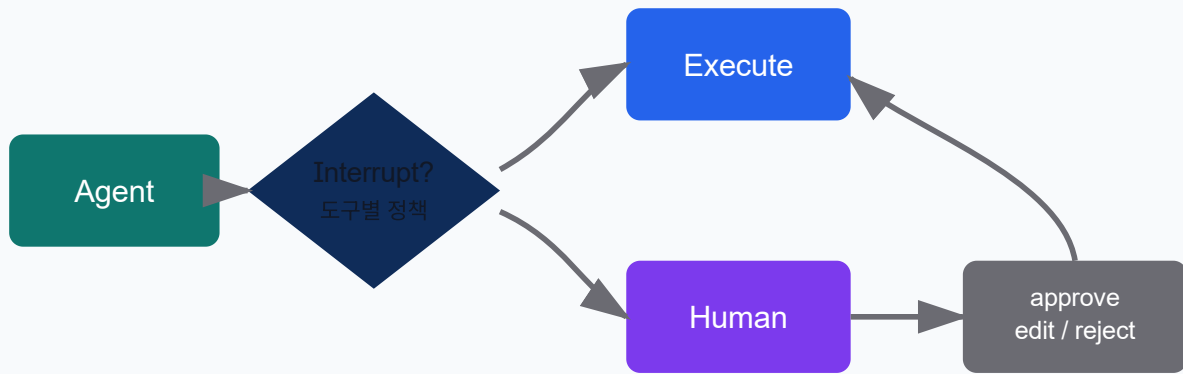
두 번째 패턴은 **여러 전문 서브에이전트**다. 예를 들어 API 정확성을 검토하는 `api-reviewer` 와 발표 문구를 다듬는 `slide-writer` 를 나누면, 각 에이전트의 도구와 출력 형식을 좁힐 수 있다. 메인 에이전트는 설명을 보고 어느 쪽에 맡길지 선택한다.

세 번째 패턴은 **CompiledSubAgent**다. 단순 딕셔너리가 아니라 이미 컴파일된 LangGraph 그래프를 서브에이전트로 제공한다. 복잡한 상태 전이, 별도 노드, 검증 단계를 가진 워크플로우가 있을 때 사용한다. 일반 발표에서는 “복잡한 그래프를 서브에이전트 슬롯에 꽂는 방식” 정도로 이해하면 충분하다.

서브에이전트 모범 사례는 간단하다. 설명은 구체적으로 쓰고, 시스템 프롬프트에는 출력 형식을 포함하며, 도구는 필요한 것만 준다. 마지막으로 서브에이전트의 반환값은 짧아야 한다. 서브에이전트가 긴 보고서를 그대로 반환하면 컨텍스트 격리의 이점이 사라진다.

§4. Human-in-the-loop 기본

Human-in-the-loop은 에이전트가 위험한 도구를 호출하려는 순간 실행을 멈추고 사람에게 결정을 맡기는 흐름이다. deepagents에서는 `interrupt_on` 으로 어떤 도구를 멈출지 지정한다.



```

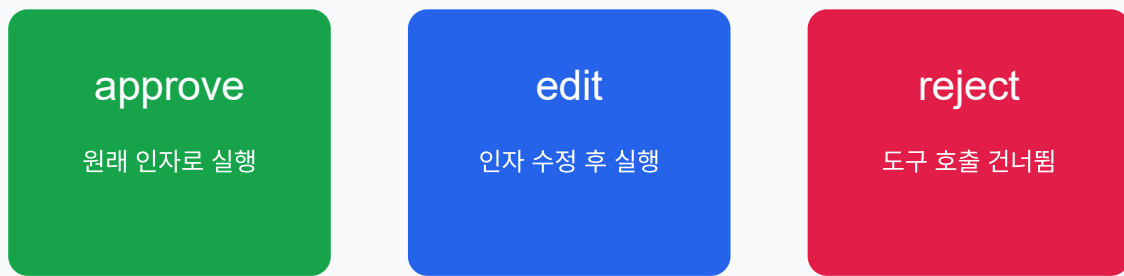
agent = create_deep_agent(
    model=model,
    tools=[delete_file, read_file, send_email],
    interrupt_on={
        "delete_file": True,
        "read_file": False,
        "send_email": {"allowed_decisions": ["approve", "reject"]},
    },
    checkpointer=MemorySaver(),
)
  
```

`interrupt_on` 값은 세 가지 형태를 가진다. `True` 는 기본 승인 결정을 모두 허용한다. `False` 는 해당 도구를 멈추지 않는다. 딕셔너리 형태는 허용할 결정을 직접 제한한다.

중요한 점은 체크포인트가 필수라는 것이다. 인터럽트는 실행을 잠시 멈춘 뒤 나중에 같은 상태에서 재개해야 한다. 그래서 `MemorySaver` 같은 체크포인트와, 호출마다 유지되는 `thread_id` 가 필요하다. 첫 호출과 재개 호출에서 `thread_id` 가 달라지면 에이전트는 멈춘 지점을 찾을 수 없다.

§5. 승인 처리

사람이 내릴 수 있는 결정은 `approve` , `edit` , `reject` 다.



결정	의미
approve	에이전트가 제안한 원래 인자로 도구 실행
edit	실행 전에 도구 이름과 인자를 수정
reject	해당 도구 호출을 건너뛰기

인터럽트가 발생하면 결과의 `__interrupt__`를 확인한다. 그 안에는 `action_requests`와 `review_configs`가 들어 있다. UI나 CLI는 이 정보를 사람에게 보여주고, 결정 목록을 만든 뒤 `Command(resume={"decisions": decisions})`로 재개한다.

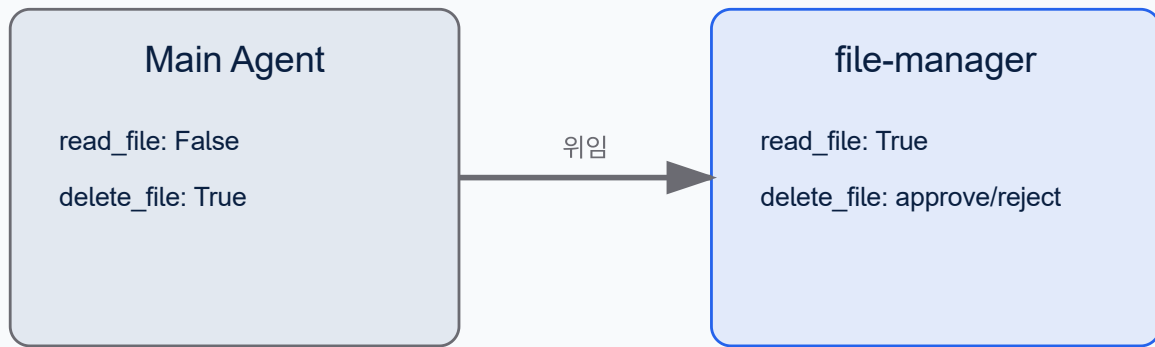
```
PYTHON

if result.get("__interrupt__"):
    interrupts = result["__interrupt__"][0].value
    action_requests = interrupts["action_requests"]
    decisions = [{"type": "approve"}]
    result = agent.invoke(Command(resume={"decisions": decisions}), config=config)
```

여러 도구 호출이 한 번에 멈추면 결정도 같은 순서로 여러 개 제공해야 한다. `edit`을 사용할 때는 `edited_action`에 도구 `name`과 수정된 `args`를 함께 넣는다. 예를 들어 전체 회사 메일을 팀 메일로 바꾸는 식이다.

§6. 서브에이전트와 승인 결합

서브에이전트도 자체 `interrupt_on`을 가질 수 있다. 메인 에이전트에서는 `read_file`을 안전하다고 보고 멈추지 않더라도, `file-manager` 서브에이전트에서는 민감 파일을 다룬다는 이유로 읽기까지 승인 대상으로 만들 수 있다.



```

PYTHON
subagents=[{
    "name": "file-manager",
    "description": "파일 읽기와 삭제를 처리한다.",
    "system_prompt": "파일 작업 요청은 적절한 도구를 호출하라.",
    "tools": [read_file, delete_file],
    "interrupt_on": {
        "read_file": True,
        "delete_file": {"allowed_decisions": ["approve", "reject"]},
    },
}]
  
```

이 패턴은 책임 경계가 다를 때 유용하다. 메인 에이전트는 전체 작업 조율에 집중하고, 파일 관리 서브에이전트는 더 보수적인 승인 정책을 가진다. 승인 처리는 동일하다. 최종 결과의 `__interrupt__` 를 확인하고, 사람이 내린 결정을 `Command` 로 재개한다.

부록 A. 트러블슈팅

증상	원인 / 해결
서브에이전트가 호출되지 않음	<code>description</code> 이 모호하거나 메인 프롬프트가 위임을 요구하지 않음
컨텍스트가 여전히 커짐	서브에이전트 반환이 너무 길거나 메인 도구가 직접 큰 결과를 읽음
인터럽트가 재개되지 않음	체크포인트가 없거나 첫 호출과 재개 호출의 <code>thread_id</code> 가 다름
<code>edit</code> 재개 실패	<code>edited_action</code> 에 도구 <code>name</code> 또는 필수 <code>args</code> 가 빠짐

부록 B. 실행 스크립트

실행 스크립트는 `scripts/README.md` 에 정리되어 있다. 01 과 02 는 서브에이전트 구성, 03 부터 05 는 HITL 승인 흐름을 다룬다.

부록 C. 참고문헌

기타

[1] `06-subagents_ko.md` — SubAgent 구성, 범용 서브에이전트, 모범 사례, 문제 해결.

[2] `07-human-in-the-loop_ko.md` — `interrupt_on` , 승인 결정, `Command(resume=...)` , 서브에이전트 인터럽트.